

A Multi-Model Framework for Short-Horizon Stock Market Risk and Investment Quality Forecasting

Ujjwal Jung Karki

March 19, 2026

Abstract

This project develops and evaluates a computational framework for short-horizon stock market forecasting with an emphasis on *risk-aware investment assessment* rather than naive price prediction. Using daily market data for 24 liquid U.S. assets spanning broad-market ETFs, sector ETFs, and large-cap equities, I constructed a panel dataset, engineered technical and market-context features, and defined multiple forward-looking targets: cumulative future return, realized volatility, downside semivolatility, a Sharpe-like score, and a binary investment-quality label. The modeling pipeline compares linear baselines, nonlinear tree-based models, and deep sequence-learning architectures.

The central objective was to determine whether sequence models—especially LSTM-based architectures with temporal attention and multi-task outputs—could outperform strong tabular baselines in forecasting short-horizon market risk and identifying favorable investment opportunities. The final results show that classical tabular models, particularly Ridge regression for volatility forecasting and Random Forest for nonlinear classification, outperform the implemented deep-learning alternatives in this feature-rich setting. This negative but informative result is itself valuable: it shows that when temporal information has already been heavily encoded through engineered rolling, technical, and market-state features, simpler models can remain highly competitive and even dominant.

Beyond equity-market screening, the same computational logic is applicable in several domains that require short-horizon risk forecasting, structured sequential prediction, and decision-aware classification. These include portfolio construction, hedging, algorithmic execution, risk monitoring, insurance and actuarial forecasting, energy-market volatility analysis, macro-financial surveillance, supply-chain risk scoring, and any setting where future performance must be judged jointly with uncertainty.

1 Introduction

Forecasting financial markets is difficult because returns are noisy, nonlinear, regime-dependent, and sensitive to both endogenous market dynamics and broader macro-financial conditions. A large portion of low-quality financial machine learning work focuses narrowly on raw price-direction prediction. Such setups are often conceptually weak because they ignore the fact that profitable investment decisions depend not only on expected return, but also on uncertainty, volatility, and downside risk.

This project was designed to address that weakness. Instead of asking, “Will tomorrow’s price go up?”, the framework asks more defensible questions:

- Can future *realized volatility* be forecast accurately at short horizons?
- Can future *investment quality* be assessed using a label that accounts for both return and risk?
- Do deep sequence models actually add predictive value once strong technical and rolling time-series features have already been engineered?

The project therefore combines three perspectives:

1. **Risk forecasting:** estimating short-horizon realized volatility and downside semivolatility.
2. **Investment screening:** classifying whether an asset appears favorable under a return-risk criterion.
3. **Model comparison:** evaluating whether neural sequence models outperform simpler linear and tree-based alternatives.

This is not merely an exercise in applying a neural network. It is a structured empirical study of when deep learning helps, when it fails, and what this implies for practical financial modeling.

2 Objectives

The main objectives of the project were:

1. To construct a multi-asset panel dataset from daily historical market data for broad-market ETFs, sector ETFs, and liquid large-cap equities.

2. To engineer a feature set combining return dynamics, rolling risk measures, technical indicators, market-wide context, and calendar information.
3. To define multiple forward-looking targets representing both *risk* and *investment attractiveness*.
4. To compare classical and modern models, specifically:
 - Ridge regression,
 - Logistic regression,
 - Random Forest regression and classification,
 - single-task LSTM with attention,
 - multi-task LSTM with attention.
5. To determine whether deep sequence models outperform tabular models in short-horizon risk-aware market forecasting.
6. To interpret the results in a manner that is practical, defensible, and relevant for broader computational finance and applied risk modeling.

3 Broader Relevance and Applications

Although this study is framed around equities and ETFs, the methodology has much wider applicability. The computational structure developed here—panel feature engineering, multi-horizon target definition, time-ordered evaluation, and comparative model testing—can be transferred to many real-world problems.

3.1 Applications in Finance

- **Portfolio screening:** ranking assets by predicted investment quality before allocation.
- **Risk management:** forecasting realized volatility and downside risk for exposure control.
- **Algorithmic trading:** filtering candidates for short-horizon tactical strategies.
- **Volatility targeting:** adjusting portfolio leverage based on predicted short-term risk.
- **Market surveillance:** identifying periods or assets likely to enter unstable regimes.
- **Execution planning:** choosing whether to increase, reduce, or stagger exposure under expected volatility.

3.2 Applications Beyond Finance

- **Energy markets:** predicting price volatility and risk-adjusted opportunity in power, gas, or carbon markets.
- **Insurance and actuarial science:** risk scoring under uncertain future outcomes.
- **Supply-chain systems:** identifying favorable versus unstable procurement opportunities.
- **Macroeconomic forecasting:** combining panel time-series predictors for short-horizon stress indicators.
- **Operational risk:** classifying favorable versus risky states in dynamic systems.

Thus, the project demonstrates a general decision-aware forecasting pipeline, not merely a stock-prediction exercise.

4 Data

4.1 Asset Universe

The analysis used 24 liquid U.S. assets:

- Broad-market ETFs: SPY, QQQ, IWM
- Sector ETFs: XLK, XLF, XLV, XLE, XLI, XLP, XLU, XLY, XLB, XLRE
- Large-cap equities: AAPL, MSFT, NVDA, AMZN, GOOGL, META, JPM, XOM, JNJ, UNH, PG

Daily adjusted data were downloaded from Yahoo Finance for the period from 2012-01-01 to 2026-01-01. The raw download contained 3,520 dates and, after reshaping into panel form, 84,480 asset-date rows.

4.2 Train/Validation/Test Split

A time-ordered split was used:

- **Training:** up to 2020-12-31

- **Validation:** 2021-01-04 to 2023-12-29
- **Test:** 2024-01-02 to 2025-12-30

This split avoids leakage and respects the temporal structure of financial data.

5 Feature Engineering

The modeling pipeline used structured tabular features designed to summarize short- and medium-horizon market behavior.

5.1 Price and Return Features

The following were derived from adjusted daily price data:

- log close price,
- one-day log return,
- one-day simple return,
- rolling means of returns over 5, 10, 20, and 60 days,
- rolling standard deviations of returns over 5, 10, 20, and 60 days,
- momentum over 5, 10, 20, and 60 days,
- high-low range,
- open-close return.

5.2 Volume Features

Volume-based signals included:

- log volume,
- one-day log-volume change,
- 20-day rolling mean of log volume.

5.3 Technical Indicators

Standard technical indicators were added for each asset:

- RSI(14),
- MACD,
- MACD signal,
- MACD difference,
- Bollinger Band width,
- ATR(14).

5.4 Market Context Features

To capture broader regime information, the following contextual features were included:

- SPY one-day return,
- SPY 20-day rolling volatility,
- SPY 20-day momentum,
- VIX level,
- VIX one-day log return.

5.5 Calendar Features

Calendar and seasonal effects were represented with:

- day of week,
- month,
- quarter,
- sinusoidal encodings of day-of-week and month.

6 Target Construction

The project explicitly avoided raw price prediction and instead defined forward-looking, risk-aware targets.

6.1 Future Return

For horizon h ,

$$\text{future_ret}_{h,t} = \sum_{i=1}^h r_{t+i}.$$

6.2 Future Realized Volatility

For horizon h ,

$$\text{future_rv}_{h,t} = \sqrt{\sum_{i=1}^h r_{t+i}^2}.$$

This target was the main regression variable of interest.

6.3 Future Downside Semivolatility

For horizon h ,

$$\text{future_dsv}_{h,t} = \sqrt{\sum_{i=1}^h \min(r_{t+i}, 0)^2}.$$

This emphasizes downside variation rather than total variation.

6.4 Future Score

A Sharpe-like forward score was defined as

$$\text{future_score}_{h,t} = \frac{\text{future_ret}_{h,t}}{\text{future_rv}_{h,t} + 10^{-8}}.$$

This is not a full Sharpe ratio in the classical sense, but a useful risk-adjusted return proxy.

6.5 Binary Investment-Quality Label

For each horizon h , a binary label was defined as:

$$\text{quality_label}_{h,t} = \begin{cases} 1, & \text{if future return is positive and future realized volatility is below the cross-section} \\ 0, & \text{otherwise.} \end{cases}$$

This formulation makes the classification task more meaningful than plain return-direction forecasting.

7 Models

7.1 Ridge Regression

Ridge regression was used as a linear baseline for forecasting 5-day realized volatility. It tests whether a penalized linear combination of engineered predictors is already sufficient to explain short-horizon risk dynamics.

7.2 Logistic Regression

Logistic regression with class balancing was used as the simplest interpretable classifier for the binary investment-quality label.

7.3 Random Forest

Random Forest was used in both regression and classification settings to provide a nonlinear tabular benchmark. Since the project is feature-rich, this model serves as a strong test of whether nonlinear interactions among engineered variables are already enough.

7.4 Single-Task LSTM with Attention

A sequence model was implemented using:

- 60-day input windows,
- a 2-layer LSTM,

- temporal attention,
- a final fully connected head for 5-day realized-volatility prediction.

This model was designed to exploit temporal order directly rather than rely purely on tabular summary features.

7.5 Multi-Task LSTM with Attention

Two multitask variants were trained:

- an original multi-task LSTM predicting `future_rv_5`, `future_dsv_5`, `future_score_5`, and `quality_label_5`,
- a fixed multitask LSTM with standardized regression targets, lower hidden dimension, stronger regularization, and early stopping.

The motivation was to test whether jointly learning related tasks improves representation quality.

8 Evaluation Metrics

8.1 Regression Metrics

For regression targets:

- RMSE,
- MAE,
- R^2 .

8.2 Classification Metrics

For the binary investment-quality task:

- Accuracy,
- Precision,

- Recall,
- F1-score,
- ROC-AUC.

These metrics allow separate analysis of predictive sharpness, classification balance, and ranking quality.

9 Results

9.1 Volatility Forecasting Results

Table 1 summarizes the 5-day realized-volatility results on the test set.

Table 1: Test-set performance for forecasting 5-day realized volatility

Model	RMSE	MAE	R^2
Ridge Regression	0.008681	0.004951	0.821895
Random Forest	0.008695	0.004640	0.821335
Single-Task LSTM	0.014407	0.008702	0.532521
Original Multi-Task LSTM	0.019268	0.011923	0.163782
Fixed Multi-Task LSTM	0.017240	0.010538	0.330593

The key observation is that Ridge and Random Forest perform almost identically and far outperform the deep-learning variants. This suggests that the engineered tabular feature space already captures most of the predictive structure relevant for 5-day volatility.

The single-task LSTM improves on both multitask variants but remains substantially weaker than the tabular baselines. The fixed multitask model improves materially over the original multitask model, which indicates that target standardization and architectural simplification matter, but the improvement is not enough to challenge the simpler methods.

9.2 Binary Investment-Quality Classification Results

Table 2 shows the test-set performance for the binary quality label.

Table 2: Test-set performance for investment-quality classification

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.737797	0.550589	0.799882	0.652226	0.832199
Random Forest	0.790873	0.631528	0.767414	0.692871	0.862590
Original Multi-Task LSTM	0.754071	0.636086	0.427397	0.511266	0.775162
Fixed Multi-Task LSTM	0.708308	0.520344	0.394178	0.448558	0.719722

Random Forest is the best classifier overall, with the highest accuracy, F1-score, and ROC-AUC. Logistic regression performs surprisingly well and remains competitive. The multi-task LSTM variants underperform both tabular baselines. While the original multitask model had moderate classification accuracy, both multitask variants exhibit weak recall-precision balance relative to simpler baselines.

9.3 Additional Multi-Task Regression Results

The multi-task models were also evaluated on the auxiliary regression tasks:

- `future_dsv_5`,
- `future_score_5`.

For the fixed multi-task model, the test-set results were:

- `future_dsv_5`: RMSE = 0.015852, MAE = 0.009892, $R^2 = 0.256032$
- `future_score_5`: RMSE = 0.966681, MAE = 0.799609, $R^2 = 0.063167$

These values indicate that the multitask network struggled to learn a robust shared representation across all tasks, especially for the score variable.

10 Discussion

10.1 Why Did the Simple Models Win?

The strongest empirical conclusion from the project is that classical tabular models outperformed the deep sequence models in this setting. There are several plausible reasons.

First, the feature-engineering stage was already rich and temporally informed. The models did not receive only raw price data; they received rolling returns, rolling volatility, momentum, RSI, MACD, ATR, Bollinger width, market context from SPY, and VIX variables. Once that information is made explicit, a linear or tree-based model can often exploit it effectively without needing to infer temporal structure from scratch.

Second, the forecasting horizon was short. At a 5-day horizon, short-term risk may be better summarized by recent rolling statistics than by a flexible sequence model that has many degrees of freedom and limited effective signal.

Third, multi-task learning introduced optimization difficulty. The multitask models had to balance regression and classification objectives simultaneously, and even after standardizing the regression targets, the auxiliary tasks appear to have contaminated the shared representation rather than improved it.

10.2 Interpretation of the Neural Results

The neural models did not outperform, but they remain useful in the report because they establish a credible comparison. This turns the project into an empirical investigation rather than a one-model demonstration.

The single-task LSTM shows that temporal modeling is not automatically superior to strong feature engineering. The original multi-task LSTM demonstrates that naïve multi-task learning can degrade performance. The fixed multitask model demonstrates that thoughtful corrections improve results but do not necessarily reverse the ranking.

This makes the report more defensible, not less. A graduate-level project is stronger when it presents an honest comparative finding rather than forcing a neural method to appear successful.

10.3 Practical Implication

If the goal is short-horizon risk-aware asset screening in a structured feature setting, the current evidence suggests:

- use Ridge or Random Forest for realized-volatility forecasting,
- use Random Forest for investment-quality classification,
- use deep sequence models only if the feature set is redesigned, the task is reformulated, or the data structure is expanded.

11 Limitations

Several limitations should be acknowledged.

1. **Feature dependence:** the models rely heavily on engineered market and technical features. This helps performance but may limit generalization to settings with different market microstructure or data availability.
2. **No transaction-cost analysis:** the project assesses predictive quality and investment screening, not fully implementable trading profitability.
3. **No walk-forward retraining loop in production form:** although the split is time-ordered, a full rolling retrain framework would be an important extension.
4. **Label simplicity:** the binary investment-quality label is useful and interpretable, but still a simplified abstraction of investment decision quality.
5. **Model scope:** XGBoost and more specialized sequence architectures were not included in the final stable comparison.

12 Future Work

Several extensions naturally follow from this work:

- Implement XGBoost as a stronger nonlinear tabular benchmark.
- Add rolling-origin or walk-forward retraining.
- Evaluate ranking-based portfolio construction from predicted quality probabilities.
- Replace the current binary label with a stricter composite signal involving return, realized volatility, and downside risk.
- Explore lighter two-task sequence models, such as jointly predicting `future_rv_5` and `quality_label_5`.
- Expand the universe to a broader equity panel.
- Incorporate regime indicators or macro covariates beyond VIX.

13 Conclusion

This project developed a structured, risk-aware framework for short-horizon stock market prediction using a panel of ETFs and large-cap equities. Instead of focusing on raw price direction, the study targeted realized volatility, downside risk, and investment quality. The resulting comparison delivered a clear empirical message: in a feature-rich financial setting, simpler tabular models can outperform more complex deep-learning alternatives.

Ridge regression and Random Forest achieved the strongest 5-day realized-volatility forecasts, while Random Forest also produced the best investment-quality classification results. Single-task and multi-task LSTM-attention models did not surpass these baselines, and multitask learning proved especially difficult to stabilize. Nevertheless, these negative neural results are informative and strengthen the overall analysis by showing that model complexity alone does not guarantee superior performance.

The project is therefore defensible on both technical and conceptual grounds. It demonstrates careful dataset design, leakage-aware evaluation, risk-aware target construction, meaningful baseline comparisons, and honest interpretation of empirical outcomes. Just as importantly, it establishes a transferable computational framework that can be applied to finance, risk management, energy systems, insurance, and other domains where forecasting under uncertainty must support actual decision-making.

Appendix A: Reproducibility Notes

- Language: Python
- Core libraries: `pandas`, `numpy`, `scikit-learn`, `PyTorch`, `yfinance`, `ta`
- Environment snapshot:
 - NumPy 1.26.4
 - pandas 2.2.3
 - torch 2.2.2
- Device used for the reported run: CPU

Appendix B: Figures

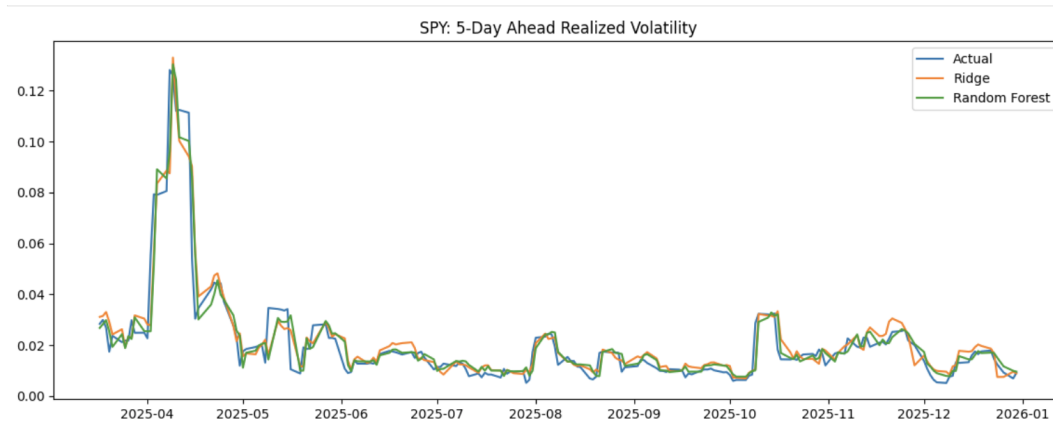


Figure 1: Actual versus predicted 5-day realized volatility for SPY using Ridge and Random Forest.

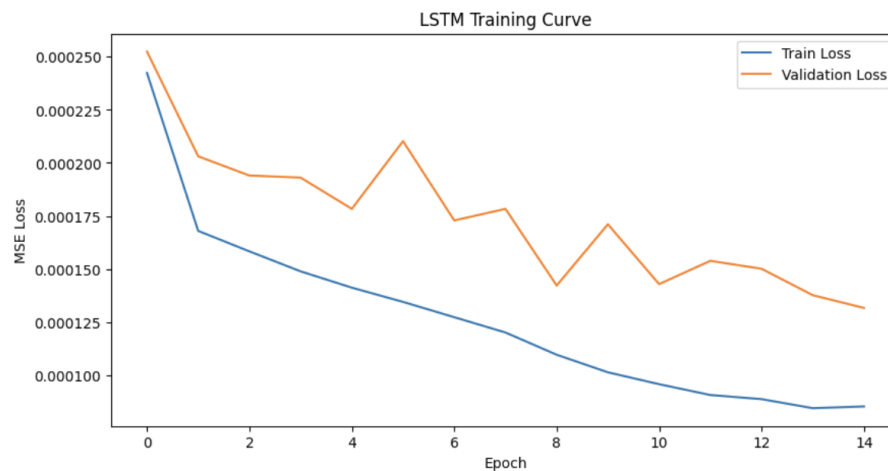


Figure 2: Training and validation loss for the single-task LSTM.

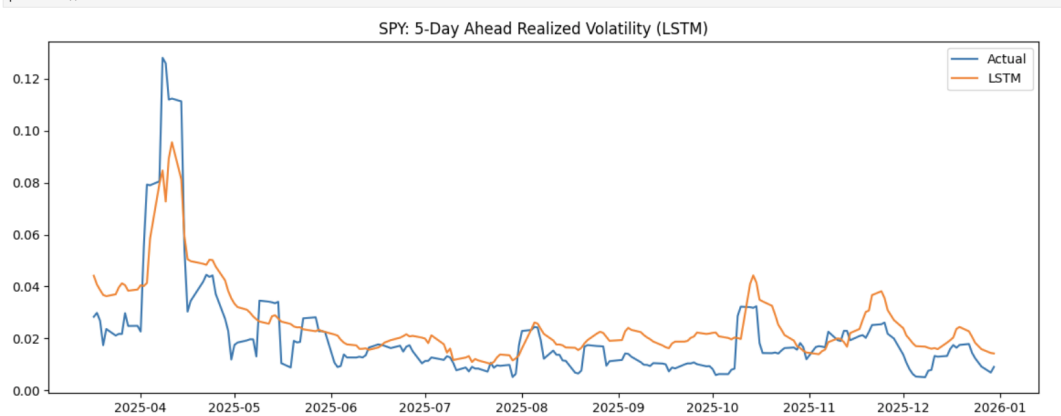


Figure 3: Actual versus predicted 5-day realized volatility for SPY using the single-task LSTM.

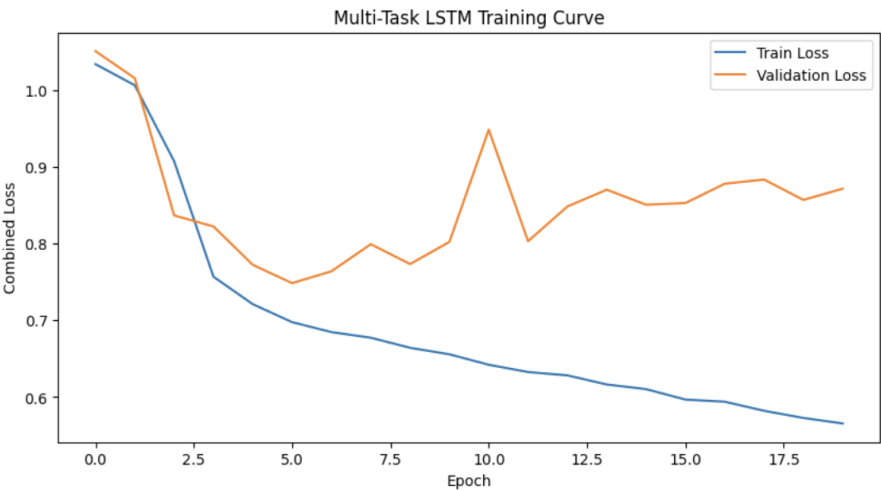


Figure 4: Training and validation loss for the original multi-task LSTM.

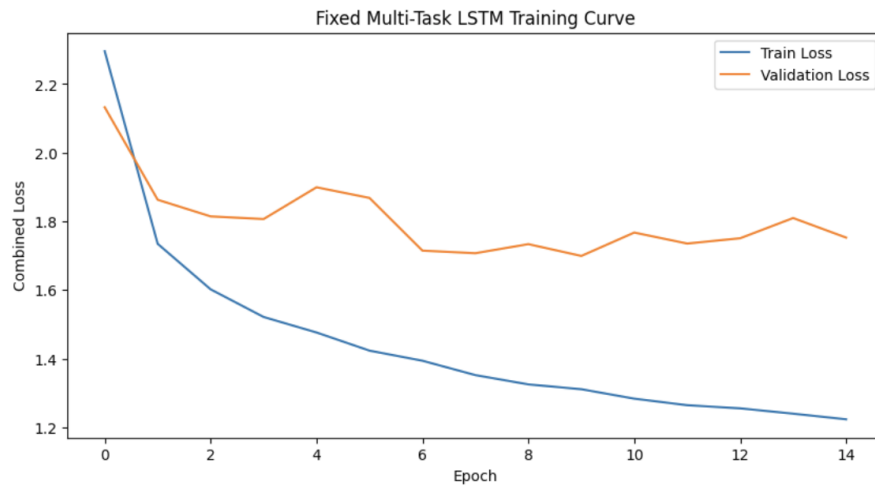


Figure 5: Training and validation loss for the fixed multi-task LSTM.

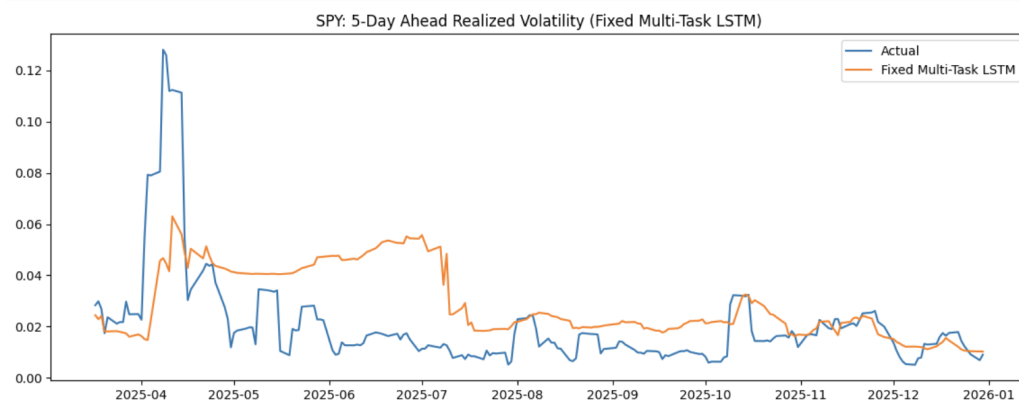


Figure 6: Actual versus predicted 5-day realized volatility for SPY using the fixed multi-task LSTM.

Appendix C: Minimal Code Snippet for Core Setup

```

1 tickers = [
2     "SPY", "QQQ", "IWM",
3     "XLK", "XLF", "XLV", "XLE", "XLI", "XLP", "XLU", "XLY", "XLB", "XLRE",
4     "AAPL", "MSFT", "NVDA", "AMZN", "GOOGL", "META", "JPM", "XOM", "JNJ",
5     "UNH", "PG"
6 ]
7 start_date = "2012-01-01"
8 end_date = "2026-01-01"
9
10 raw = yf.download(

```

```
11     tickers ,  
12     start=start_date ,  
13     end=end_date ,  
14     auto_adjust=True ,  
15     progress=False ,  
16     group_by="ticker"  
17 )
```